

Subiecte de examen oral

Partea I: Structuri de date • Partea II: Algoritmi

Partea I — Structuri de date

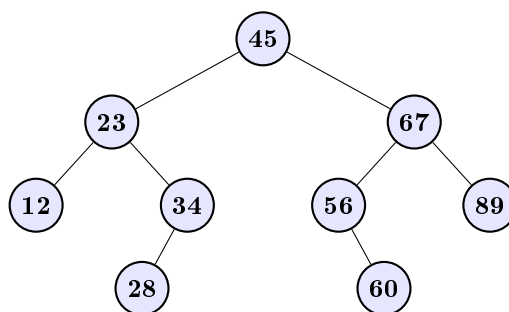
Biletul 1. Se inserează, în ordine, valorile

45 23 67 12 34 56 89 28 60

într-un **arbore binar de căutare** inițial vid. Desenați arborele obținut și scrieți parcurgerile în **preordine**, **inordine** și **postordine**.

Rezolvare.

Regula de inserare: valorile mai mici merg în subarborele stâng, cele mai mari în cel drept. Fiecare valoare coboară de la rădăcină până găsește o poziție liberă.



- **Preordine** (rădăcină, stâng, drept): 45, 23, 12, 34, 28, 67, 56, 60, 89
 - **Inordine** (stâng, rădăcină, drept): 12, 23, 28, 34, 45, 56, 60, 67, 89 — *observație:* pentru un ABC, inordinea produce mereu șirul sortat;
 - **Postordine** (stâng, drept, rădăcină): 12, 28, 34, 23, 60, 56, 89, 67, 45.
-

Biletul 2. Folosind o **stivă**, convertiți expresia din forma **infix** în forma **postfix** (poloneză inversă):

$$(a + b) * c - d / (e + f)$$

apoi evaluați expresia postfix pentru $a=2$, $b=3$, $c=4$, $d=10$, $e=1$, $f=4$.

Rezolvare.

Conversie (algoritmul shunting-yard): operanzii merg direct la ieșire; operatorii se pun pe stivă, scoțând întâi operatorii de precedență $\geq$; paranteza deschisă se pune pe stivă, iar la paranteza închisă se descarcă stiva până la „(“.

simbol	stivă	ieșire
(	(	
a	(	a
+	(+	a
b	(+	a b
)	—	a b +
*	*	a b +
c	*	a b + c
—	—	a b + c *
d	—	a b + c * d
/	— /	a b + c * d
(	— / (	a b + c * d
e	— / (	a b + c * d e
+	— / (+	a b + c * d e
f	— / (+	a b + c * d e f
)	— /	a b + c * d e f +
sfârșit	—	a b + c * d e f + / —

Postfix: a b + c * d e f + / —

Evaluare cu stivă (operand $\Rightarrow$ push; operator $\Rightarrow$ pop 2, calculează, push): $2\ 3 + \rightarrow 5$; $5\ 4 * \rightarrow 20$; $1\ 4 + \rightarrow 5$; $10\ 5 / \rightarrow 2$; $20\ 2 - \rightarrow 18$.

Biletul 3. Se dă o **coadă circulară** cu 5 poziții (indecși 0–4), implementată pe un vector, cu indicatorii **front** și **rear**. Simulați secvența de operații:

enq(3), enq(7), enq(1), deq, enq(9), deq, enq(4), enq(8), enq(5)

indicând conținutul vectorului și valorile indicatorilor după fiecare operație.

Rezolvare.

Principiu FIFO: enq scrie la **rear** și avansează $\text{rear} = (\text{rear} + 1) \bmod 5$; deq citește de la **front** și avansează $\text{front} = (\text{front} + 1) \bmod 5$.

operație	front	rear	vector (_ = liber)
inițial	0	0	[_,_,_,_,_]
enq(3)	0	1	[3,_,_,_,_]
enq(7)	0	2	[3,7,_,_,_]
enq(1)	0	3	[3,7,1,_,_]
deq $\rightarrow 3$	1	3	[_,7,1,_,_]
enq(9)	1	4	[_,7,1,9,_]
deq $\rightarrow 7$	2	4	[_,_,1,9,_]
enq(4)	2	0	[_,_,1,9,4] (rear revine la 0 — circularitate)
enq(8)	2	1	[8,_,1,9,4]
enq(5)	2	2	[8,5,1,9,4] (coada e plină)

Conținutul cozii, în ordinea de ieșire: 1, 9, 4, 8, 5. Avantajul circularității: pozițiile eliberate prin deq sunt refolosite, fără deplasarea elementelor.

Biletul 4. Construiți un **max-heap** pornind de la vectorul

4 10 3 5 1 8 9 2

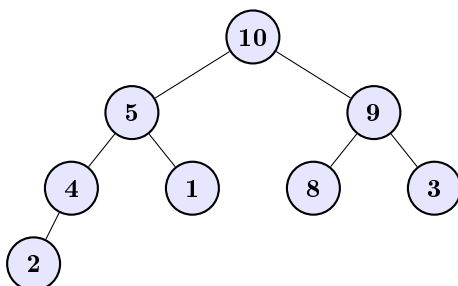
folosind metoda bottom-up (heapify), apoi efectuați două operații de **extragere a maximumului**.

Rezolvare.

Proprietate max-heap: $a[i] \geq a[2i+1]$ și $a[i] \geq a[2i+2]$ (părintele $\geq$ copiii). **Heapify bottom-up:** se aplică *sift-down* pe nodurile interne, de la ultimul ($i = n/2 - 1 = 3$) către rădăcină. Complexitate $O(n)$.

- $i=3$ (val. 5, copil 2): ok — nimic de făcut;
- $i=2$ (val. 3, copii 8, 9): schimbă cu 9 $\rightarrow [4, 10, 9, 5, 1, 8, 3, 2]$;
- $i=1$ (val. 10, copii 5, 1): ok;

- $i=0$ (val. 4, copii 10, 9): schimbă cu 10, apoi cu 5 $\rightarrow [10, 5, 9, 4, 1, 8, 3, 2]$.



Extract-max (rădăcina $\leftrightarrow$ ultimul element, micșorează heap-ul, sift-down din rădăcină, $O(\log n)$):

1. extrage **10**: 2 urcă în rădăcină, coboară $\rightarrow [9, 5, 8, 4, 1, 2, 3]$;
2. extrage **9**: 3 urcă în rădăcină, coboară $\rightarrow [8, 5, 3, 4, 1, 2]$.

Repetând extragerea se obține **HeapSort**, $O(n \log n)$.

Partea II — Algoritmi

Biletul 5. Explicați metoda **Greedy** rezolvând *problema spectacolelor*: dintre spectacolele de mai jos, alegeți numărul maxim de spectacole care pot fi vizionate integral (fără suprapuneri).

spectacol	A	B	C	D	E	F	G	H
început	1	3	0	5	6	8	8	12
sfârșit	4	5	6	7	10	11	12	14

Rezolvare.

Principiu Greedy: la fiecare pas se face alegerea optimă local, fără revenire. Pentru spectacole, criteriul corect este **ora de sfârșit minimă** (lasă cel mai mult timp liber pentru restul).

Pași (spectacolele sunt deja sortate după sfârșit):

1. **A**(1–4) — selectat; ultimul sfârșit = 4;
2. **B**(3–5): începe la $3 < 4 \Rightarrow$ respins; **C**(0–6): $0 < 4 \Rightarrow$ respins;
3. **D**(5–7): $5 \geq 4$ — selectat; ultimul sfârșit = 7;
4. **E**(6–10): $6 < 7 \Rightarrow$ respins;
5. **F**(8–11): $8 \geq 7$ — selectat; ultimul sfârșit = 11;
6. **G**(8–12): $8 < 11 \Rightarrow$ respins;
7. **H**(12–14): $12 \geq 11$ — selectat.

Soluția: {**A, D, F, H**} — 4 spectacole (maxim posibil).

Observație: alte criterii greedy (durata minimă, începutul minim) NU dau mereu optimul; criteriul sfârșitului minim se demonstrează corect prin tehnica schimbului. Complexitate: $O(n \log n)$ (sortarea).

Biletul 6. Explicați metoda **Divide et Impera** folosind *căutarea binară* a valorii $x = 23$ în vectorul sortat

2 5 8 12 16 23 38 56 72 91

Rezolvare.

Schema generală: problema se *împarte* în subprobleme mai mici de același tip, acestea se rezolvă (de regulă recursiv), iar soluțiile se *combină*. La căutarea binară, compararea cu mijlocul elimină jumătate din vector la fiecare pas — combinarea este trivială.

Trace (indecși 0–9):

1. $lo=0, hi=9, mid=4: a[4]=16 < 23 \Rightarrow$ căutăm în dreapta, $lo=5$;
2. $lo=5, hi=9, mid=7: a[7]=56 > 23 \Rightarrow$ căutăm în stânga, $hi=6$;
3. $lo=5, hi=6, mid=5: a[5]=23 \Rightarrow$ **găsit la indexul 5**.

Complexitate: recurența $T(n) = T(n/2) + O(1)$ are soluția $T(n) = O(\log n)$ — doar 3 comparații pentru $n = 10$, față de până la 10 la căutarea liniară.

Alte exemple de Divide et Impera: MergeSort și QuickSort ($T(n) = 2T(n/2) + O(n) = O(n \log n)$), turnurile din Hanoi, exponențierea rapidă.

Biletul 7. Explicați metoda **programării dinamice** rezolvând *problema rucsacului* (0/1): un rucsac suportă greutatea maximă $G = 8$; se dau obiectele

obiect	o_1	o_2	o_3	o_4
greutate	2	3	4	5
valoare	3	4	5	6

Determinați valoarea maximă transportabilă.

Rezolvare.

Principiu DP: problema are *subprobleme suprapuse* și *substructură optimă*; se memorează soluțiile subproblemelor într-un tabel. Fie $dp[i][g]$ = valoarea maximă folosind primele i obiecte la capacitate g :

$$dp[i][g] = \max(dp[i-1][g], dp[i-1][g - g_i] + v_i \text{ dacă } g \geq g_i)$$

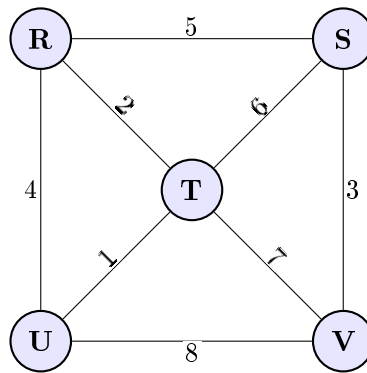
Tabelul (i pe linii, $g = 0..8$ pe coloane):

$i \backslash g$	0	1	2	3	4	5	6	7	8
0 (nimic)	0	0	0	0	0	0	0	0	0
1 (o_1)	0	0	3	3	3	3	3	3	3
2 (o_2)	0	0	3	4	4	7	7	7	7
3 (o_3)	0	0	3	4	5	7	8	9	9
4 (o_4)	0	0	3	4	5	7	8	9	10

Valoarea maximă $= dp[4][8] = 10$. *Reconstrucție:* $dp[4][8] \neq dp[3][8] \Rightarrow o_4$ luat (rămâne $g=3$); $dp[3][3] = dp[2][3] = 4 \neq dp[1][3] \Rightarrow o_2$ luat (rămâne $g=0$). **Soluția:** $\{o_2, o_4\}$, **greutate** $3 + 5 = 8$, **valoare** $4 + 6 = 10$.

Complexitate: $O(n \cdot G)$ timp. *Atenție:* greedy după raportul valoare/greutate ar alege întâi o_1 (raport 1,5) și ar obține doar $3 + 6 = 9$ — de aceea e necesară programarea dinamică.

Biletul 8. Se dă graful ponderat de mai jos. Folosind algoritmul **Kruskal**, aflați arborele parțial de cost minim.



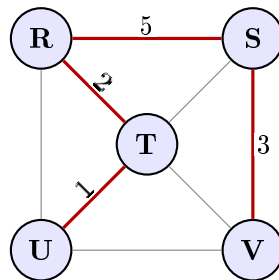
Rezolvare.

Muchii sortate crescător: $TU=1$, $RT=2$, $SV=3$, $RU=4$, $RS=5$, $ST=6$, $TV=7$, $UV=8$.

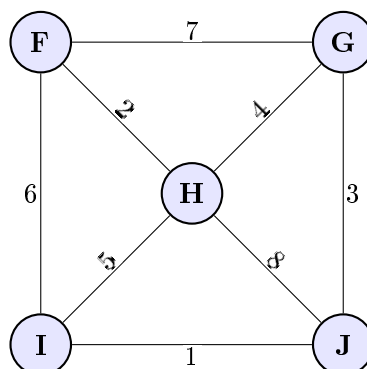
Selecție (Union-Find):

1. $TU(1)$ — adaugă. $\{T, U\}$
2. $RT(2)$ — adaugă. $\{R, T, U\}$
3. $SV(3)$ — adaugă. $\{R, T, U\}, \{S, V\}$
4. $RU(4)$ — ciclu (R, T, U deja conexe). *Respins.*
5. $RS(5)$ — adaugă (unește componentele). $\{R, S, T, U, V\}$

APM = $\{TU, RT, SV, RS\}$, **cost total** = $1 + 2 + 3 + 5 = 11$.



Biletul 9. Se dă graful ponderat de mai jos. Folosind algoritmul **Prim**, aflați arborele parțial de cost minim, pornind din vârful F .

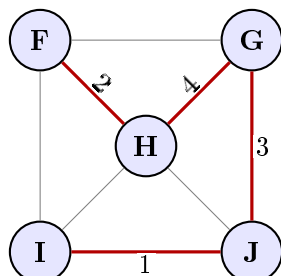


Rezolvare.

Pași (se alege mereu muchia minimă dintre vizitat și nevizitat):

1. Start $\{F\}$. Candidate: $FG=7, FH=2, FI=6 \Rightarrow \mathbf{FH}(2)$.
2. $\{F, H\}$. Candidate: $FG=7, FI=6, GH=4, HI=5, HJ=8 \Rightarrow \mathbf{GH}(4)$.
3. $\{F, G, H\}$. Candidate: $FI=6, HI=5, GJ=3, HJ=8 \Rightarrow \mathbf{GJ}(3)$.
4. $\{F, G, H, J\}$. Candidate: $FI=6, HI=5, IJ=1 \Rightarrow \mathbf{IJ}(1)$.

APM = $\{\mathbf{FH}, \mathbf{GH}, \mathbf{GJ}, \mathbf{IJ}\}$, cost total = $2 + 4 + 3 + 1 = 10$.



Biletul 10. Descrieți și exemplificați **SelectionSort**, folosind ca exemplu șirul

13 6 2 10 15 5 1 9 14 4 12 7

Rezolvare.

Principiu: la pasul i , se caută minimul din $a[i..n-1]$ și se interschimbă cu $a[i]$. Complexitate $O(n^2)$, instabil, in-place.

Trace pentru [13, 6, 2, 10, 15, 5, 1, 9, 14, 4, 12, 7]:

pas	șir după interschimbare
1: min=1@6	[<u>1</u> , 6, 2, 10, 15, 5, 13, 9, 14, 4, 12, 7]
2: min=2@2	[1, <u>2</u> , 6, 10, 15, 5, 13, 9, 14, 4, 12, 7]
3: min=4@9	[1, 2, <u>4</u> , 10, 15, 5, 13, 9, 14, 6, 12, 7]
4: min=5@5	[1, 2, 4, <u>5</u> , 15, 10, 13, 9, 14, 6, 12, 7]
5: min=6@9	[1, 2, 4, 5, <u>6</u> , 10, 13, 9, 14, 15, 12, 7]
6: min=7@11	[1, 2, 4, 5, 6, 7, <u>13</u> , 9, 14, 15, 12, 10]
7: min=9@7	[1, 2, 4, 5, 6, 7, 9, <u>13</u> , 14, 15, 12, 10]
8: min=10@11	[1, 2, 4, 5, 6, 7, 9, 10, <u>12</u> , 15, 14, 13]
9: min=12@10	[1, 2, 4, 5, 6, 7, 9, 10, 12, <u>15</u> , 14, 13]
10: min=13@11	[1, 2, 4, 5, 6, 7, 9, 10, 12, 13, <u>14</u> , 15]
11: min=14@10	[1, 2, 4, 5, 6, 7, 9, 10, 12, 13, 14, <u>15</u>]

Biletul 11. Descrieți și exemplificați **QuickSort**, folosind ca exemplu șirul

13 6 2 10 15 5 1 9 14 4 12 7

Rezolvare.

Principiu: se alege un pivot, se partiționează șirul (stânga $\leq$ pivot, dreapta $>$ pivot), apoi se recurs pe cele două jumătăți. Complexitate medie $O(n \log n)$, pesim $O(n^2)$, in-place.

Pivotul = **ultimul element** (schema Lomuto). Pe [13, 6, 2, 10, 15, 5, 1, 9, 14, 4, 12, **7**]:

Partiționare cu pivot **7** (scanare j , index i al ultimului $\leq$ pivot):

- mutări succesive: 6, 2, 5, 1, 4 devin prefixul ≤ 7 ;
- final: [6, 2, 5, 1, 4, 7, 10, 9, 14, 15, 12, 13], pivot la poziția 5.

Recurs pe stânga [6, 2, 5, 1, 4], pivot 4 $\rightarrow$ [2, 1, 4, 6, 5]; apoi [2, 1], pivot 1 $\rightarrow$ [1, 2] și [6, 5], pivot 5 $\rightarrow$ [5, 6]. Rezultat: [1, 2, 4, 5, 6].

Recurs pe dreapta [10, 9, 14, 15, 12, 13], pivot 13 $\rightarrow$ [10, 9, 12, 13, 14, 15]; pe [10, 9, 12], pivot 12 $\rightarrow$ [10, 9, 12], apoi [10, 9], pivot 9 $\rightarrow$ [9, 10]; pe [14, 15], pivot 15 $\rightarrow$ [14, 15].

Final: [1, 2, 4, 5, 6, 7, 9, 10, 12, 13, 14, 15].

Biletul 12. Se dau secvențele

BANANAREPUBLIC

ANAREBANPUBLIC

Explicați cum funcționează **LCS** (longest common subsequence) folosind cele 2 secvențe.

Rezolvare.

Principiu DP: fie $X = x_1 \dots x_m$, $Y = y_1 \dots y_n$. Definim $L[i][j] = |LCS(X_i, Y_j)|$.

$$L[i][j] = \begin{cases} 0 & i = 0 \text{ sau } j = 0 \\ L[i-1][j-1] + 1 & x_i = y_j \\ \max(L[i-1][j], L[i][j-1]) & x_i \neq y_j \end{cases}$$

Apoi se reconstruiește LCS prin *backtracking* din $L[m][n]$: dacă $x_i = y_j$, se ia caracterul și se merge diagonal; altfel către celula maximă (sus/stânga).

Aplicație: $X = \text{BANANAREPUBLIC}$ ($m = 14$), $Y = \text{ANAREBANPUBLIC}$ ($n = 14$).

Cel mai lung subșir comun: **ANAREPUBLIC** de lungime **11**.

Justificare rapidă: **ANARE** se regăsește la poziția 2 în X (din **bANAnAREpublic**) și la poziția 1 în Y , iar **PUBLIC** la poziția 9 în X și 9 în Y . Caracterul **B** inițial din X nu poate fi păstrat fără a pierde secvența **ANARE**.

Complexitate: $O(mn)$ timp și spațiu.

Biletul 13. Explicați algoritmul **Huffman** plecând de la textul:

„ave Caesar morituri te salutant”.

Rezolvare.

Principiu: codare prefix-free de lungime variabilă. Literele frecvente primesc coduri scurte. Se construiește un arbore binar prin combinarea repetată a celor mai mici două frecvențe într-un nod intern cu suma frecvențelor.

Frecvențe în „ave Caesar morituri te salutant” (31 caractere):

char	a		t	e	r	i	s	u	C	l	m	n	o	v
frec	5	4	4	3	3	2	2	2	1	1	1	1	1	1

Pași (pseudocod cu min-heap):

1. se introduc toate cele 14 noduri-frunză cu frecvențele lor;
2. cât timp sunt ≥ 2 noduri: extrage cele 2 cu frecvență minimă, creează un nod intern cu suma, reintroduce-l;
3. codul fiecărei litere = drumul rădăcină→frunză (0 stânga, 1 dreapta).

Literele rare (**C, l, m, n, o, v** cu frecvență 1) vor avea coduri de 5 biți; cele mai frecvente (**a, space, t, e, r**) vor avea 3 biți, iar **i, s, u** câte 4 biți. Lungimea totală = 111 biți, vs. $31 \cdot \lceil \log_2 14 \rceil = 124$ biți în codare fixă.

Biletul 14. Se dă textul

„dura lex sed lex”

și șablonul

„lex”

Folosind datele de mai sus explicați algoritmul **Rabin-Karp**, lucrând în baza 16. Pentru codificare se pot asocia cifrele cu literele pe baza ordinii apariției în text.

Rezolvare.

Codificare în ordinea apariției: $d=0, u=1, r=2, a=3, sp=4, l=5, e=6, x=7, s=8$.

Hash tipar „lex” = $lex = 5, 6, 7$: $h_P = 5 \cdot 16^2 + 6 \cdot 16 + 7 = 1280 + 96 + 7 = \mathbf{1383}$.

Fereastra inițială „dur” = $0, 1, 2$: $h = 0 + 16 + 2 = 18$. $\neq 1383$.

Rolling hash: $h' = (h - c_{ies} \cdot 16^2) \cdot 16 + c_{intra}$.

- „ura” = $1, 2, 3$: $h = (18 - 0) \cdot 16 + 3 = 291$.
- „ra_” = $2, 3, 4$: $h = (291 - 256) \cdot 16 + 4 = 564$.
- „a_l” = $3, 4, 5$: $h = (564 - 512) \cdot 16 + 5 = 837$.
- „_le” = $4, 5, 6$: $h = (837 - 768) \cdot 16 + 6 = 1110$.
- **poziția 5:** „lex” = $5, 6, 7$: $h = 1383 \Rightarrow$ *coincidență de hash*. Verificare caracter-cu-caracter $\Rightarrow$ **potrivire**.
- ... (se continuă fereastra cu fereastră)
- **poziția 13:** „lex”: $h = 1383 \Rightarrow$ verificare $\Rightarrow$ **potrivire**.

Tiparul apare de **2 ori**: la **pozițiile 5 și 13**. Complexitate medie $O(n + m)$; pesim $O(nm)$ (multe coliziuni).

Biletul 15. Se dă textul
„mamamia mama mia”

și șablonul

„mama”

Folosind datele de mai sus explicați algoritmul **Knuth-Morris-Pratt**.

Rezolvare.

Principiu: se preprocesează tabelul *fail* (cea mai lungă bordură proprie a fiecărui prefix al tiparului), iar la nepotrivire se sare înainte fără a rebobina textul.

Tabelul *fail* pentru „mama”:

poziție	0	1	2	3
caracter	m	a	m	a
fail[k]	0	0	1	2

Scanare în „mamamia mama mia” (lungime 16):

- $i=0..3$: toate se potrivesc $\Rightarrow$ **potrivire la poziția 0**; $j \leftarrow \text{fail}[3] = 2$.
- $i=4$: m = tipar[2] $\Rightarrow j=3$; la $i=5$ i vs. a, nepotrivire $\Rightarrow j \leftarrow \text{fail}[2] = 1$, apoi fail[0] = 0, i avansează.
- $i=8..11$: „mama” se potrivește $\Rightarrow$ **potrivire la poziția 8**; $j \leftarrow 2$.
- $i=12$: spațiu vs. m, $j \leftarrow 0$; restul textului („mia”) nu mai produce potriviri.

Tiparul apare de **2 ori**: la **poziția 0 și 8**. Complexitate $O(n + m)$.